WebDelivery — Documentazione di Progetto

Corso: Web Engineering — A.A. 2025/2026, Specifica 1 **Autori:** Alessia De Dominicis [matr. 295438], Pier Marco Graziani [matr. 266107] **Repository:** https://github.com/dedoale/WebDelivery---Web-Engineering-Project

---

Indice

1. Panoramica
2. Dipendenze software
3. Funzionalità realizzate e non realizzate
4. Navigation Diagram
5. Schema relazionale del Database
6. Layout del sito
7. Tecnologie avanzate utilizzate
8. Compatibilità browser
9. Screenshot delle pagine principali
10. Validazione HTML5 (W3C Markup Validation)
11. Contributo dei componenti del gruppo

---

1. Panoramica

WebDelivery è l'interfaccia web di un'attività di ristorazione con servizio di consegna a domicilio. Il sito gestisce tre tipologie di utenza — **clienti**, **personale** e **proprietario** — ciascuna con una propria area e un proprio livello di accesso ai dati, secondo quanto richiesto dalla specifica di progetto.

Il backend è realizzato in **Java** con **servlet Jakarta EE**, il templating lato server usa **Freemarker**, la persistenza è su **MySQL**. Il frontend è **HTML5 + CSS3 + JavaScript vanilla**, senza framework client-side.

---

2. Dipendenze software

2.1 Lato server (Maven, vedi `pom.xml`)

Libreria	Versione	Motivazione
`jakarta.servlet-api`	(Jakarta EE 6.0)	API servlet, richiesta dalla specifica
`com.mysql:mysql-connector-j`	8.3.0	Driver JDBC per la connessione al DBMS MySQL
`org.freemarker:freemarker`	2.3.32	Template engine lato server per la generazione delle pagine HTML

Libreria	Versione	Motivazione
`no.api.freemarker:freemarker-java8`	2.0.0	Estensione di Freemarker per il supporto nativo a `java.time.LocalDateTime`/`LocalDate`
`jakarta.mail-api` + `org.eclipse.angus:angus-mail`	2.1.3 / 2.0.3	Invio delle email di riepilogo ordine e di notifica "in consegna" tramite SMTP
`com.google.code.gson:gson`	2.11.0	Serializzazione/deserializzazione JSON lato server

Nessun application server è incluso nel progetto: è richiesto un servlet container Jakarta EE 10-compatibile (es. Tomcat 10+) e un'istanza MySQL raggiungibile con le credenziali indicate in `web.xml` (`db.url`, `db.username`, `db.password`).

Per il test dell'invio email è previsto un server SMTP locale fittizio (es. FakeSMTP) in ascolto sulla porta `2525`, come suggerito dalla specifica.

2.2 Lato client

Nessuna dipendenza di terze parti. Tutto il codice client (`js/*.js`) è JavaScript vanilla; il layout è CSS puro, senza framework né font o icone caricate da CDN esterni. Usa solo font di sistema (`ui-monospace`/`Courier New` per i titoli, `Segoe UI`/`system-ui` per il testo).

---

3. Funzionalità realizzate e non realizzate

3.1 Funzionalità realizzate

Area pubblica / anonima

- Consultazione pubblica del menù, filtrabile per categoria, senza necessità di autenticazione.
- Dal menù è possibile visualizzare il nome, la descrizione, il prezzo, l'immagine e le possibili modifiche dei vari prodotti.
- Pagine pubbliche di presentazione dell'attività ("chi siamo", "dove siamo").
- Registrazione libera come cliente, con inserimento obbligatorio di telefono e indirizzo.
- Login/logout, gestione sessione.

Area cliente

- Composizione dell'ordine: selezione prodotti, scelta delle caratteristiche (incluse quelle di default e i gruppi a mutua esclusione, con eventuale opzione di default pre-selezionata), carrello persistito **lato server in sessione**.
- Calcolo in tempo reale del prezzo totale e del tempo di preparazione totale (somma dei tempi di preparazione dei singoli prodotti) durante la composizione del carrello.
- Tempo di consegna di default nel checkout impostato a orario attuale + tempo di preparazione totale + tempo di consegna stimato se l'ordine viene effettuato durante orario di apertura, altrimento è impostato a orario di inizio della prossima apertura + tempo di preparazione totale + tempo di consegna stimato.

- Possibilità di selezionare un orario di consegna nel checkout, con validazione **sia lato client che lato server** che impedisce di scegliere un orario fuori dagli orari di apertura o antecedente al minimo consentito: orario attuale + tempo di preparazione totale + tempo di consegna stimato oppure orario di inizio della prossima apertura + tempo di preparazione totale + tempo di consegna stimato se il locale è chiuso al momento dell'ordine.
- Ricalcolo lato server del prezzo totale e tempo di preparazione.
- Invio di un'email di riepilogo alla conferma dell'ordine, e di una seconda email quando l'ordine passa allo stato "In Consegna" (invio asincrono, non bloccante per l'utente.
- Degradazione graceful senza JavaScript delle funzionalità vitali: visualizzazione menù, comporre il carrello, effettuare il login, scegliere l'orario di consegna e confermare l'ordine. Con JavaScript disattivato rimane comunque possibile effettuare un ordine, seppur con delle feature perse (per esempio selezionare le modifiche dei piazzi, l'aggiornamento counter elementi nel carrello in altro a destra, ecc).
- Storico ordini del cliente, distinzione tra ordini "in corso" e "storico" con gli ordini già consegnati. In entrambi i casi si può visualizzare lo storico dello stato di avanzamento di ciascun ordine.

Area personale

- Lista degli ordini correnti, filtrabile per stato.
- Possibilità di avanzare lo stato di un ordine **solo in sequenza progressiva** e di un solo stato alla volta (vincolo verificato lato server).
- Tracciamento di quale membro del personale ha effettuato ciascun cambio di stato con data e ora.
- Visualizzazione scheda completa di un prodotto ordinato, compresi anche i dati non visibili al cliente (ingredienti, tempo di preparazione e procedura).

Area proprietario

- Statistiche per il proprietario: incasso giornaliero e mensile (selezionabili per data), classifica dei 3 prodotti più venduti e dei 3 meno venduti nel giorno selezionato.
- Vista di tutti gli ordini, filtrati per stato, con cronologia di quale membro del personale ha effettuato un cambio di stato e data e ora delle modifiche.
- Gestione completa del menu: creazione, modifica e eliminazione **soft**, inoltre può inserire/modificare tutti i dati relativi a un prodotto, compresi l'immagine e le caratteristiche per le personalizzazioni.
- L'eliminazone di un prodotto dal menù è soft-delete, viene cioè reso "inattivo" (per non perdere lo storico ordini) e quindi non più visualizzato.
- Vista "cestino" con i prodotti eliminati di recente e possibilità di ripristino (vengono "riattivati").
- Gestione del personale: registrazione di nuovi membri, rimozione membri esistenti (implementata come "declassamento" del ruolo).

Sicurezza

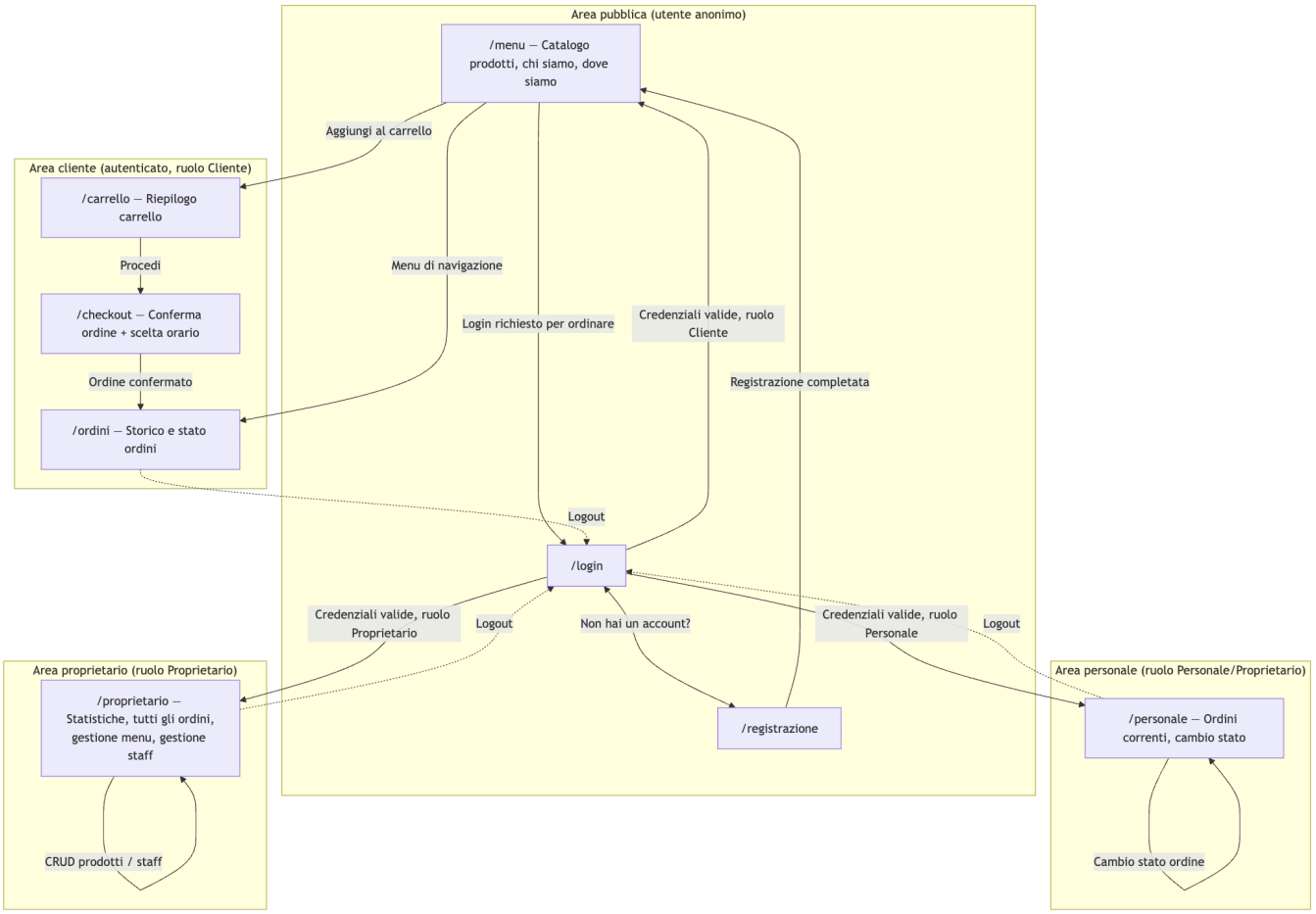
- Hashing delle password: SHA-256 salted e 10.000 iterazioni.
- Tutte le query del database usano `PreparedStatement` per impedire le SQL injection.
- Controllo ruoli centralizzato su ogni endpoint riservato.

3.2 Funzionalità non realizzate / parzialmente realizzate

- **Verifica dell'indirizzo email in fase di registrazione** tramite link di conferma: non implementata. La registrazione è immediata dopo l'invio del form.
- **Paginazione del menù**. Il menù è attualmente organizzato per categoria ma non paginato.
- **Stima del tempo di consegna basata sulla distanza reale** dall'indirizzo del cliente. Attualmente si usa un tempo di percorrenza fisso (10 minuti); il codice client (checkout.js) è già isolato in una costante dedicata proprio in previsione di questa ipotetica estensione futura.
- **Layout responsive**: Il sito è pensato principalmente per viewport da desktop/tablet.

---

4. Navigation Diagram

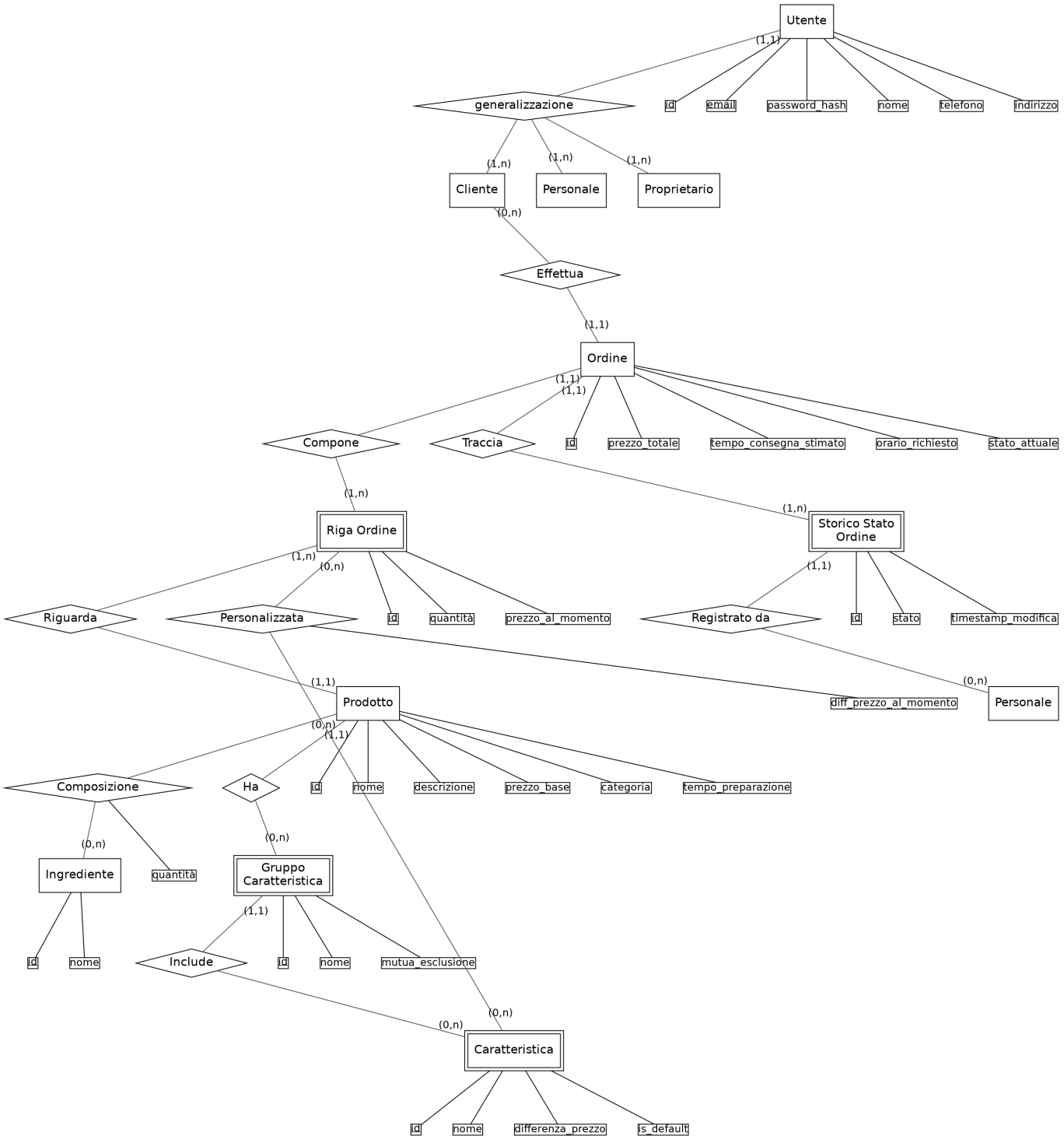


Note sulla navigabilità

- L'accesso a `/carrello`, `/checkout`, `/ordini` è riservato agli utenti autenticati con ruolo `Cliente`; l'accesso a `/personale` è consentito a `Personale` e `Proprietario`; `/proprietario` è riservato esclusivamente al `Proprietario`. Il controllo è centralizzato in `BaseServlet.verificaRuoli`.
- Un tentativo di accesso a una pagina protetta senza sessione attiva reindirizza automaticamente a `/login`.
- Endpoint dati JSON di supporto (non pagine navigabili): `GET /api prodotti` (usato da `menu.js`) e `GET /api/ordini` (usato da `orders.js`).

---

5. Schema relazionale del Database



Note di progettazione

- I prezzi (`prezzo_base_al_momento` in `Riga_Ordine`, `diff_prezzo_al_momento` in `Variante_Riga_Ordine`) vengono **congelati al momento dell'ordine**: se in seguito il proprietario cambia il prezzo di un prodotto o di una caratteristica, gli ordini già effettuati non vengono alterati retroattivamente.
- Il soft delete su `Prodotto`, `Gruppo_Caratteristica` e `Caratteristica` (`is_active`) evita di dover eliminare in cascata righe d'ordine storiche che referenziano quel prodotto o quelle caratteristiche e di compromettere lo `Storico_Stato_Ordine`.
- Il soft delete è implementato anche per i membri del personale: non vengono mai cancellati fisicamente, ma "declassati" a Utente-Eliminato, per non perdere la tracciabilità in `Storico_Stato_Ordine`.

- `Storico_Stato_Ordine` traccia lo stato, il membro del personale che lo ha impostato e l'orario della modifica.
- La mutua esclusione tra caratteristiche è modellata a livello di `Gruppo_Caratteristica.mutua_esclusione`.

Nel progetto sono inclusi Script SQL completi: `sql/01_create_tables.sql` (per iniializzare lo schema) e `sql/02_insert_data.sql` (dati di esempio per popolarlo).

---

6. Layout del sito

Il layout adotta uno stile "da ticket da cucina/comanda", coerente col tema food-delivery, con una palette a contrasto alto (inchiostro scuro su sfondo chiaro, accenti rosso "timbro" e verde "cucina") definita tramite CSS custom properties in `css/style.css` e riutilizzata in tutti i fogli di stile specifici di sezione.

6.1 Struttura Freemarker

- `base.ftl` — macro `layout.page`, usata da tutti gli altri file `.ftl` (`index.ftl`, `login.ftl`, `registrazione.ftl`, `carrello.ftl`, `checkout.ftl`, `personale.ftl`, `proprietario.ftl`, `storico-ordini.ftl`): header, navigazione, footer comuni, con uno slot `customScripts` per includere lo script specifico di pagina.
- Tutte le viste condividono gli stessi fogli di stile comuni (`style.css`, `orders.css`) più uno specifico per sezione (`auth.css`, `checkout.css`, `staff.css`, `proprietario.css`).

6.2 Componenti statiche (renderizzate da Freemarker)

- Header, footer, navigazione principale.
- Contenuto del menu pubblico (`index.ftl`): elenco prodotti e categorie.
- Le statistiche aggregate del proprietario (incasso, prodotti più/meno richiesti): calcolate server-side in `ProprietarioServlet` e renderizzate direttamente in HTML, perché sono contenuto da mostrare e non dati da manipolare via script.
- I form di autenticazione, checkout, gestione menu/staff: la struttura del form è HTML statico; funzionano anche senza JavaScript tramite submit reale verso le rispettive servlet.

6.3 Componenti dinamiche (JavaScript)

- **Filtro categoria** per i prodotti visualizzati nel menù (`menu.js`).
- **Drawer del carrello** e selezione delle caratteristiche prodotto (`menu.js`): calcolo in tempo reale di prezzo e tempo stimato mentre l'utente compone l'ordine, con sincronizzazione verso il carrello di sessione lato server tramite `fetch`.
- **Vista ordini del personale** (`staff.js`): filtri per stato, modale con scheda prodotto "interna", avanzamento di stato.
- **Dashboard del proprietario** (`proprietario.js`): tabelle dinamiche di ordini/gestione menù/staff, gestione dei campi dinamici (ingredienti, caratteristiche) nei form di creazione/modifica prodotto.
- **Storico ordini del cliente** (`orders.js`): tab "in corso"/"storico", timeline di avanzamento stato.
- Tutti e quattro questi script caricano i propri dati tramite `fetch` verso endpoint JSON dedicati invece di riceverli iniettati nel template.

---

7. Tecnologie avanzate utilizzate

7.1 Freemarker

- Freemarker come template engine lato server, per separare la generazione dell'HTML dalla logica applicativa nelle servlet.

7.2 Pattern "dati via fetch" per alcune viste

- Invece di iniettare i dati direttamente nel template Freemarker, le pagine caricano i dati in JSON da endpoint dedicati (`/api/prodotti`, `/api/ordini`, `/api/personale`) tramite `fetch()`. Questo mantiene i template Freemarker privi di logica applicativa (solo markup + un singolo `<script src="...">`) e rende i file JavaScript riutilizzabili e testabili in isolamento.

7.3 Gson per la serializzazione JSON lato server

Le viste che richiedono dati dinamici (menu con dettagli interni, ordini, elenco staff) li ricevono tramite `fetch` verso endpoint dedicati (`it.univaq.webdelivery.controller.api.*`), che serializzano gli oggetti Java in JSON con **Gson**. L'alternativa che avevamo inizialmente valutato era quella di costruire le stringe JSON manualmente, tuttavia concatenare le stringhe con escaping a mano è una fonte comune di bug (caratteri non escapati, virgole di troppo/mancanti) e abbiamo deciso di sostituirla integralmente implementando Gson. Per lo stesso motivo, anche il parsing **in ingresso** del JSON che il form di gestione prodotto invia al server (ingredienti e caratteristiche, costruiti lato client con `JSON.stringify`) viene convertito da un parser a espressioni regolari a `Gson.fromJson`.

Un'istanza `Gson` condivisa (`util/GsonProvider`) centralizza la configurazione, incluso un `TypeAdapter` per `java.time.LocalDateTime`, così il formato delle date resta coerente in tutti gli endpoint.

Dove i campi da esporre in JSON non coincidono con quelli dell'entity di dominio (es. lo staff non deve esporre `password_hash`, oppure i nomi dei campi inviati dal client non combaciano con quelli dell'entity), è stato introdotto un livello di **DTO dedicati** (`data.dto.*`) invece di serializzare/deserializzare direttamente le entity persistenti.

7.4 Assenza di framework/librerie client

Per l'ampiezza delle interazioni richieste (drawer, modali, filtri, form dinamici), abbiamo reputato sufficiente l'uso di JavaScript vanilla con DOM API standard, evitando così il peso di un framework e restando aderenti all'indicazione della specifica di non abusare di tecnologie client, soprattutto nelle parti a utenza pubblica.

7.5 Carrello in sessione lato server

Il carrello vero e proprio non vive nel browser ma in sessione HTTP (`CarrelloServlet`); il JavaScript (`menu.js`) ne mantiene solo una copia locale per un rendering rapido del drawer, sincronizzata a ogni modifica verso lo stesso endpoint usato dai `<form>` HTML. Questa implementazione garantisce che aggiungere/rimuovere un prodotto dal carrello funzioni anche a JavaScript disabilitato, per non rendere le funzionalità del front-end pubblico dipendenti dallo script.

7.6 Invio email asincrono

`EmailUtil` invia le email (riepilogo ordine, notifica "in consegna") su un `ExecutorService` separato, per non bloccare la risposta HTTP in attesa del round-trip SMTP. È predisposto per un server SMTP di test locale (porta 2525, tipo FakeSMTP) come suggerito dalla specifica.

7.7 Validazione prezzo, tempo di preparazione, orario di consegna

Ricalcolo server-side di prezzo, tempo di preparazione e validità dell'orario di consegna in fase di checkout: il client invia solo l'intenzione d'ordine (prodotti, quantità, caratteristiche scelte, orario desiderato); tutti i valori rilevanti vengono ricalcolati e validati leggendo lo stato reale del catalogo dal database, per non fidarsi ciecamente dei dati inviati dal browser.

7.8 Hashing delle password "from scratch"

- Per evitare di introdurre una dipendenza esterna dedicata (es. BCrypt) è stato implementato un semplice schema salt + SHA-256 con key stretching (10.000 iterazioni) in `PasswordUtil`, sufficiente per gli scopi didattici del progetto pur non essendo allo stato dell'arte quanto Argon2/BCrypt.

---

8. Compatibilità browser

Il sito è stato testato su:

- **Mozilla Firefox** (ultime versioni) — pienamente funzionante.
- **Google Chrome** (ultime versioni) — pienamente funzionante.
- **Safari** (ultime versioni) - pienamente funzionante.
- **Microsoft Edge** (ultime versioni) - pienamente funzionante
- **Opera** (ultime versioni) - pieanamente funzionante.
- **Google Chrome** [ver. 100 (2022)] - pieanamente funzionante.
- **Google Chrome** [ver. 37 (2014)] - JavaScript non funzionante, quindi si può effettuare login/registrazione e comporre un ordine.

---

9 Screenshot delle pagine principali

9.1 Schermate pubbliche / anonime

Pagina menu

UFD **Univaq Food Delivery** Menu Chi siamo Dove siamo Accedi Registrati 0

ORDINI APERTI

La cucina universitaria consegnata a casa tua.

Piatti preparati al momento dal nostro forno a legna. Scegli, componi, ricevi una stima del tempo prima ancora di confermare.

Vai al menu ↓

Il Menu

Ordini aggiornati in cucina in tempo reale. Tocca un piatto per personalizzarlo.

Tutte | Antipasti | Primi | Pizze | Dolci | Bevande



Bruschette miste

Quattro semplici e gustose bruschette al pomodoro, con un goccio di olio EVO e foglie di basilico fresco.

Tocca per personalizzare →

€5,50

Aggiungi al carrello



Tagliatelle al ragù

Pasta fresca all'uovo con ragù di carne cotto lentamente per 4 ore.

Tocca per personalizzare →

€9,00

Aggiungi al carrello



Pizza Margherita

La tipica pizza napoletana cotta nel forno a legna.

Tocca per personalizzare →

€7,00

Aggiungi al carrello



Pizza Diavola

L'iconica pizza piccante dal sapore deciso.

Tocca per personalizzare →

€8,00

Aggiungi al carrello



Tiramisù della casa

Dolce con savoiardi, mascarpone e caffè fatto da noi.

Tocca per personalizzare →

€4,50

Aggiungi al carrello



Caffè

Espresso della nostra torrefazione di fiducia.

Tocca per personalizzare →

€1,00

Aggiungi al carrello

Chi siamo

Univaq Food Delivery nasce nel 2026 come piccola realtà per far fronte alle esigenze culinarie degli studenti universitari fuori sede. Piatti semplici, gustosi ed economici consegnati direttamente alla tua porta di casa, caldi come al tavolo.

ORARI
Tutti i giorni
12:00–15:00
19:00–23:00

ZONA DI CONSEGNA
Entro 6 km dal locale

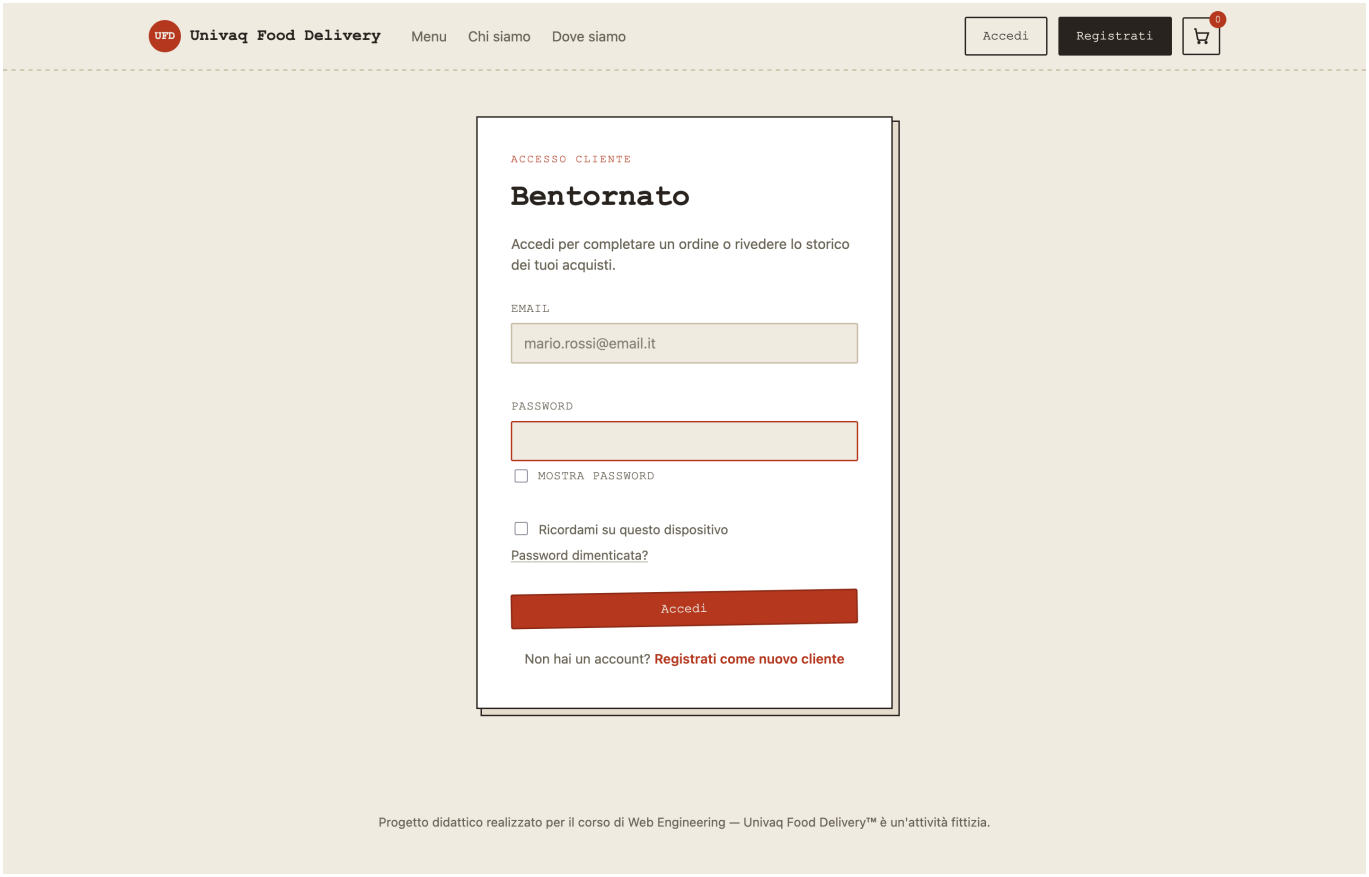
TEMPO MEDIO DI CONSEGNA
25–40 minuti

Dove siamo

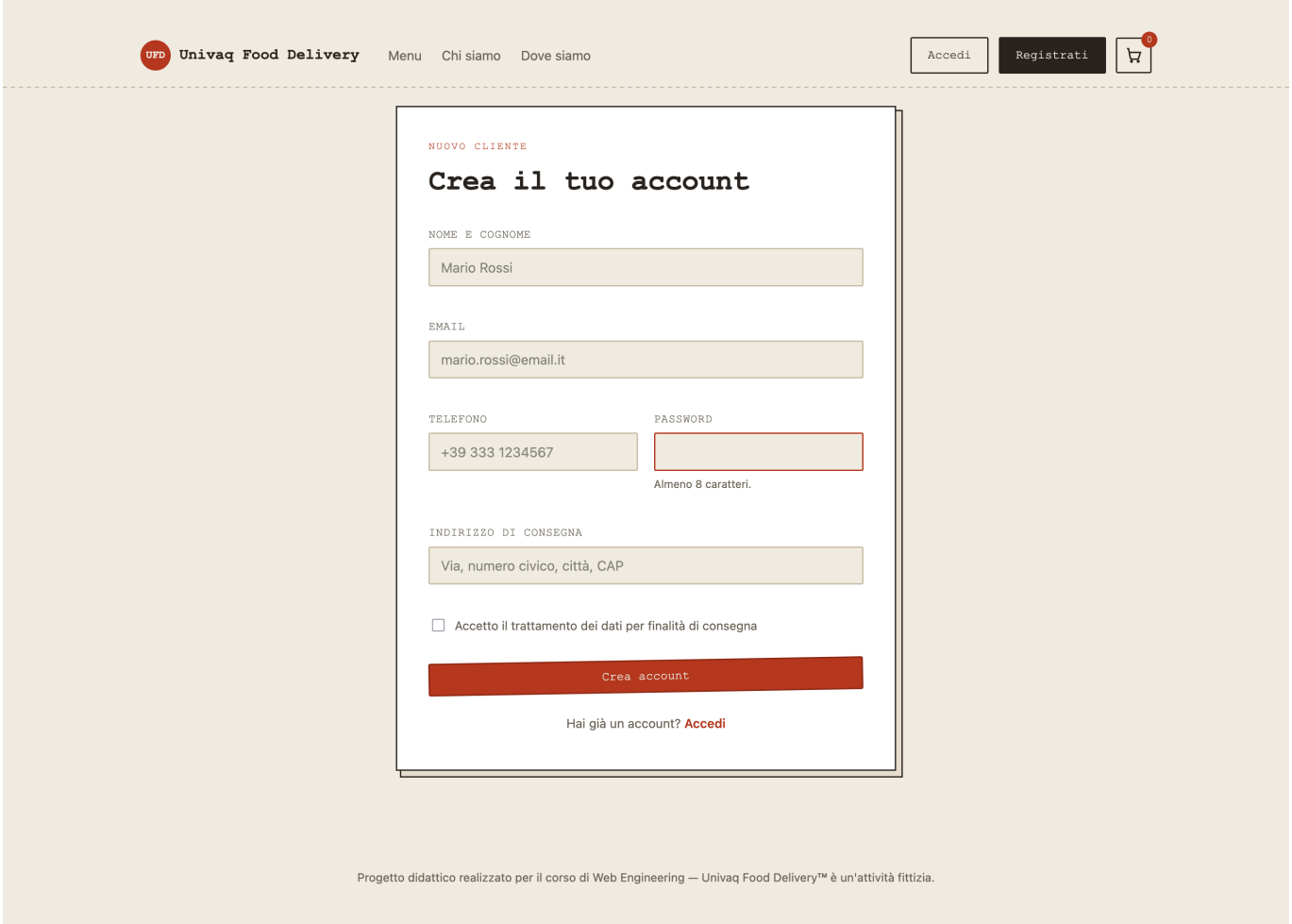
Via Vetoio — 67100 L'Aquila


Via Vetoio — 67100 L'Aquila
Progetto didattico realizzato per il corso di Web Engineering — Univaq Food Delivery™ è un'attività fittizia.

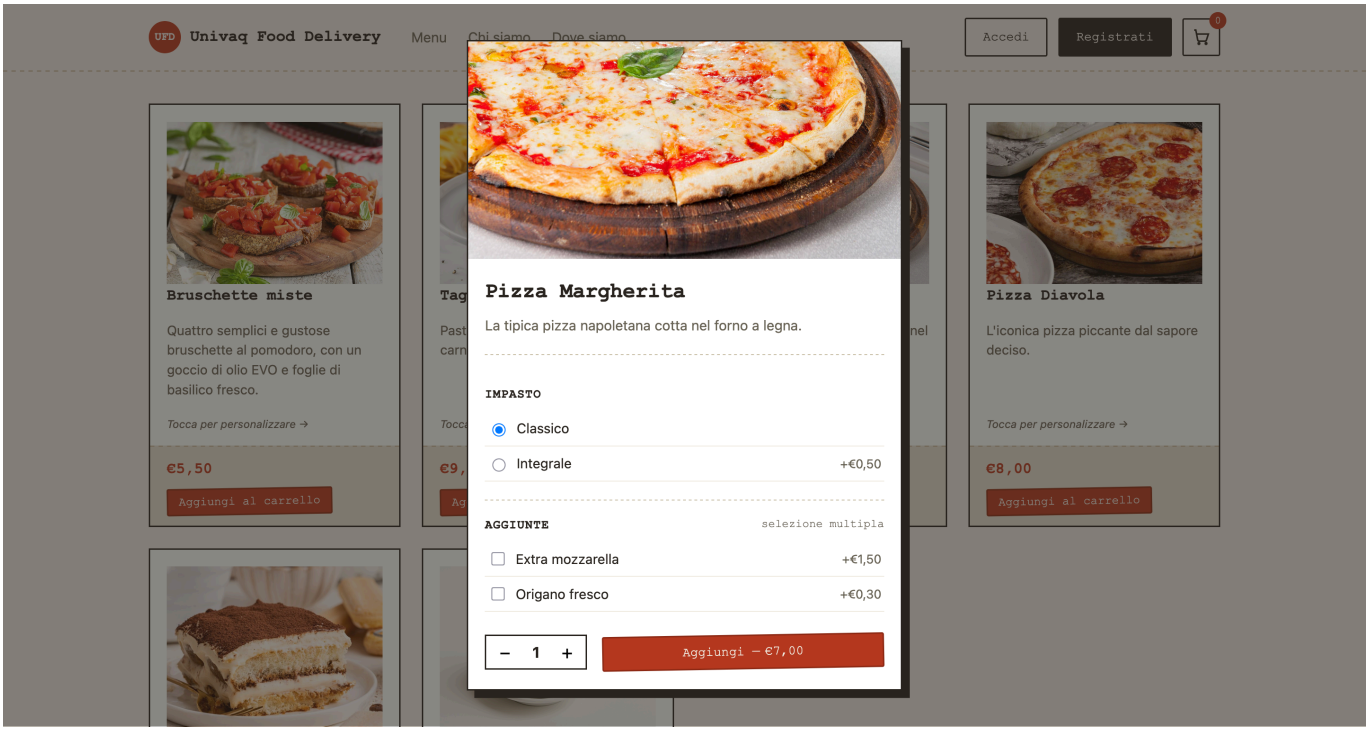
Login


UFD
Univaq Food Delivery
Menu
Chi siamo
Dove siamo
Accedi
Registrati
0
ACCESSO CLIENTE
Bentornato
Accedi per completare un ordine o rivedere lo storico dei tuoi acquisti.
EMAIL
mario.rossi@email.it
PASSWORD
MOSTRA PASSWORD
Ricordami su questo dispositivo
Password dimenticata?
Accedi
Non hai un account? Registrati come nuovo cliente
Progetto didattico realizzato per il corso di Web Engineering — Univaq Food Delivery™ è un'attività fittizia.

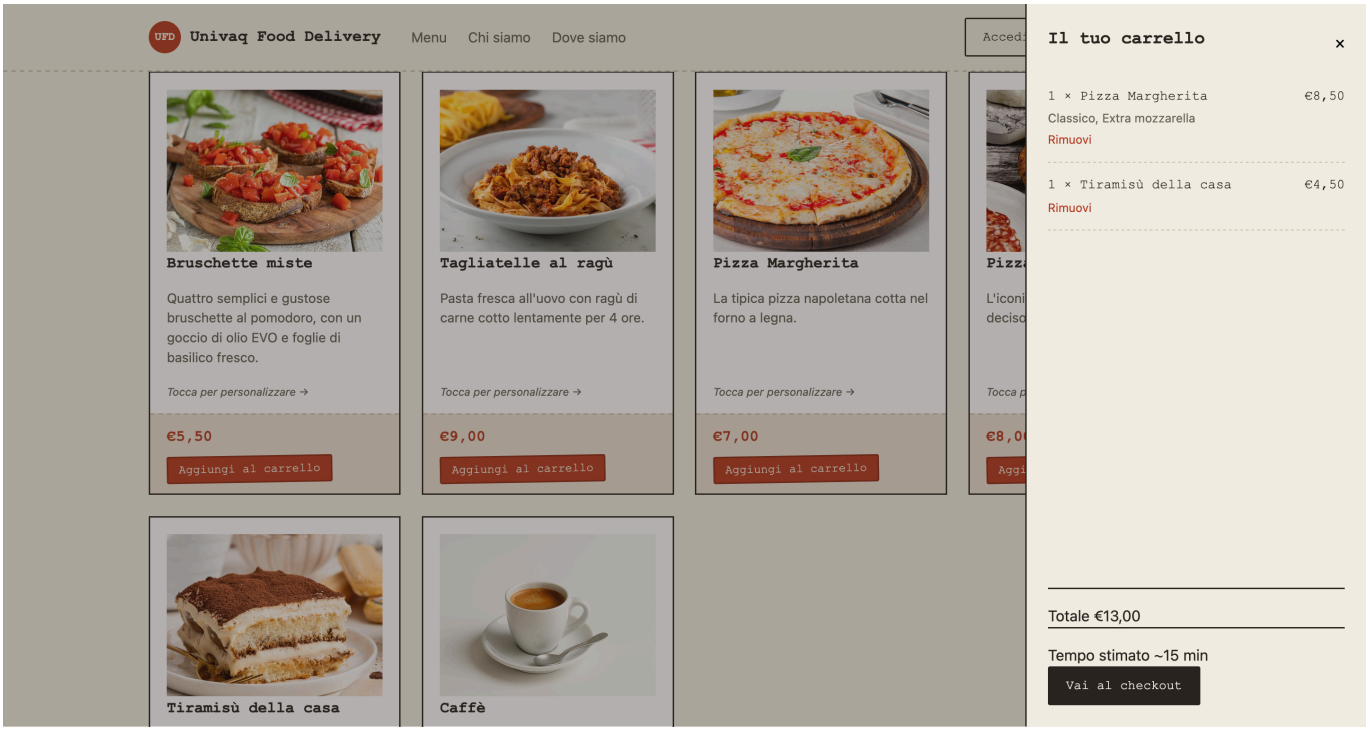
registrazione



Modale selezione modifiche di un prodotto

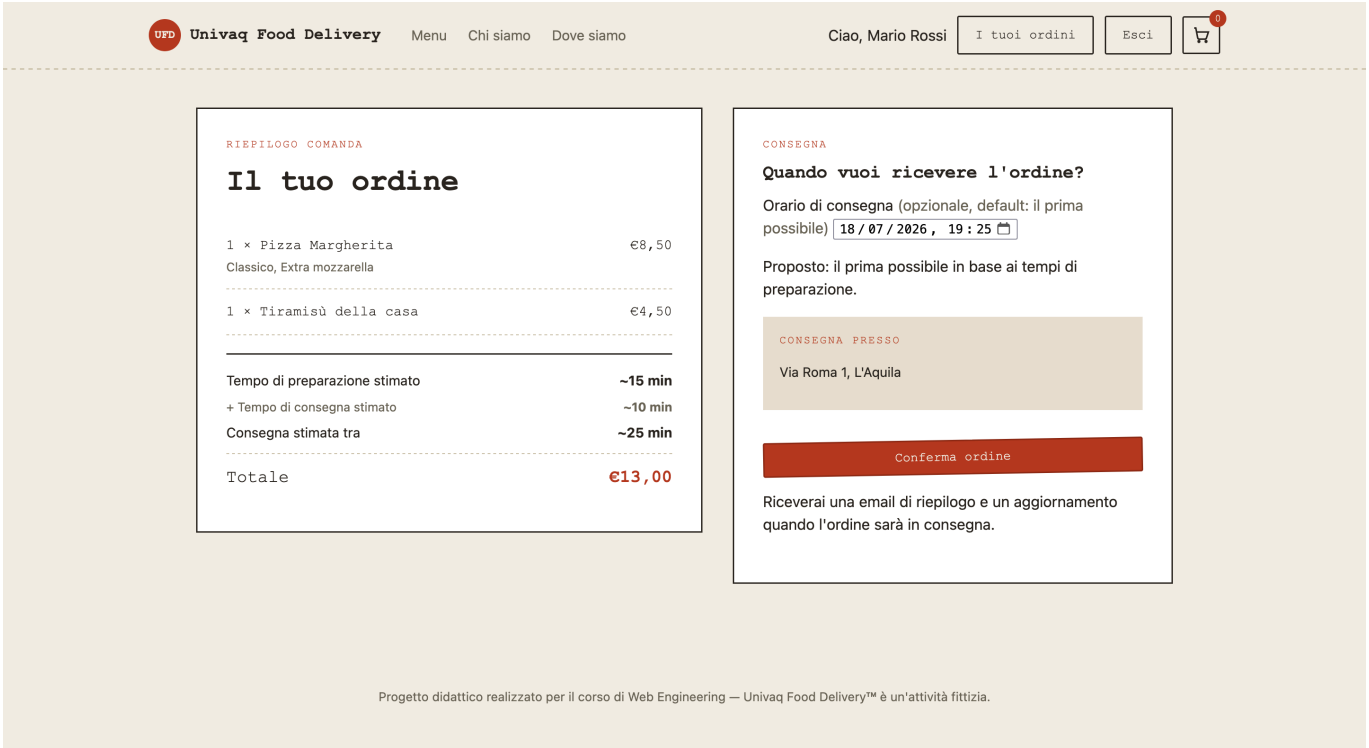


Drawer carrello

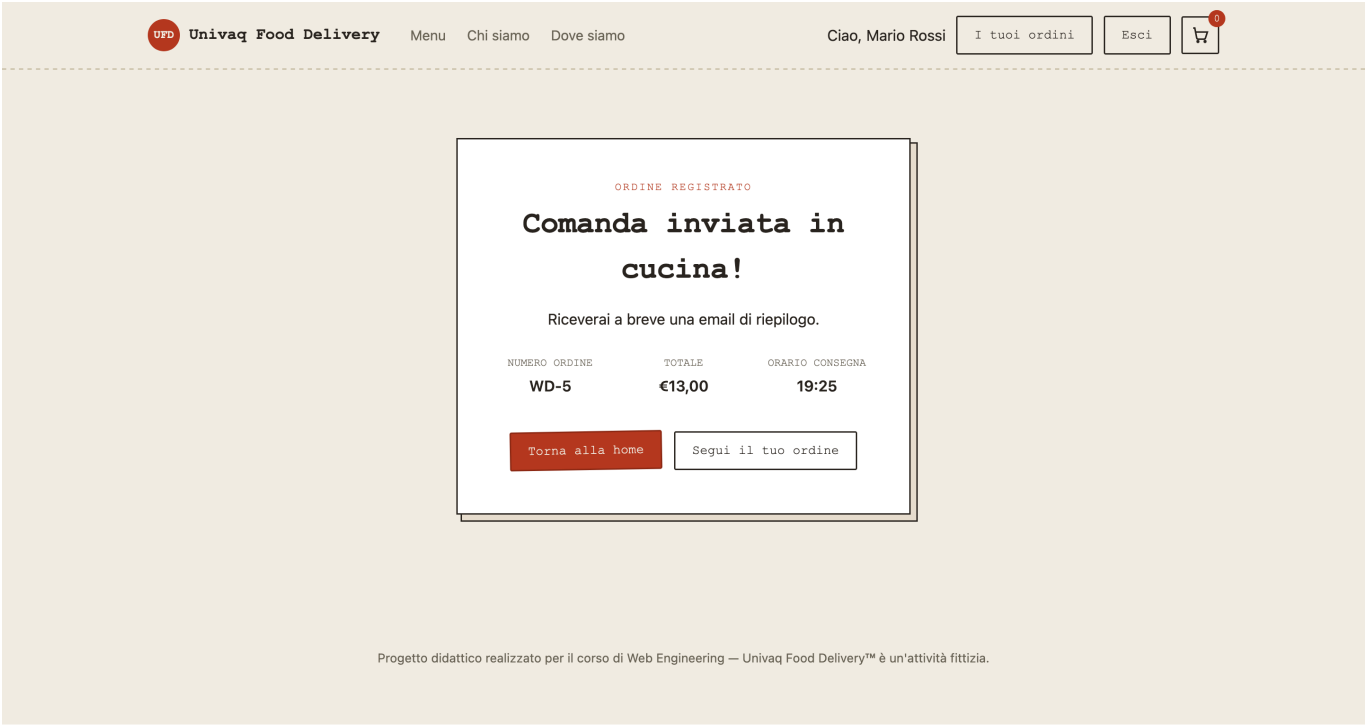


9.2 Schermate cliente

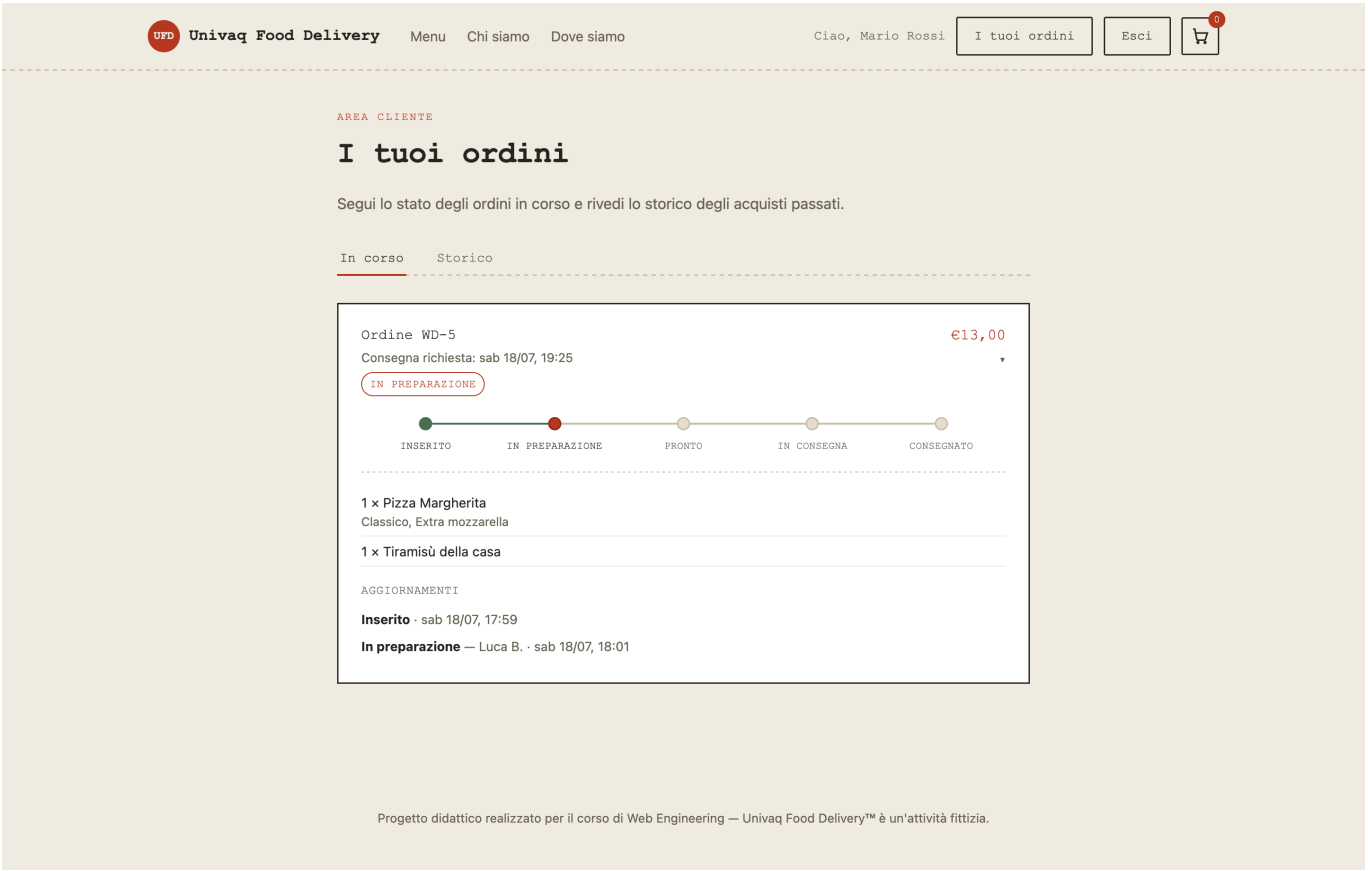
Checkout



Conferma ordine

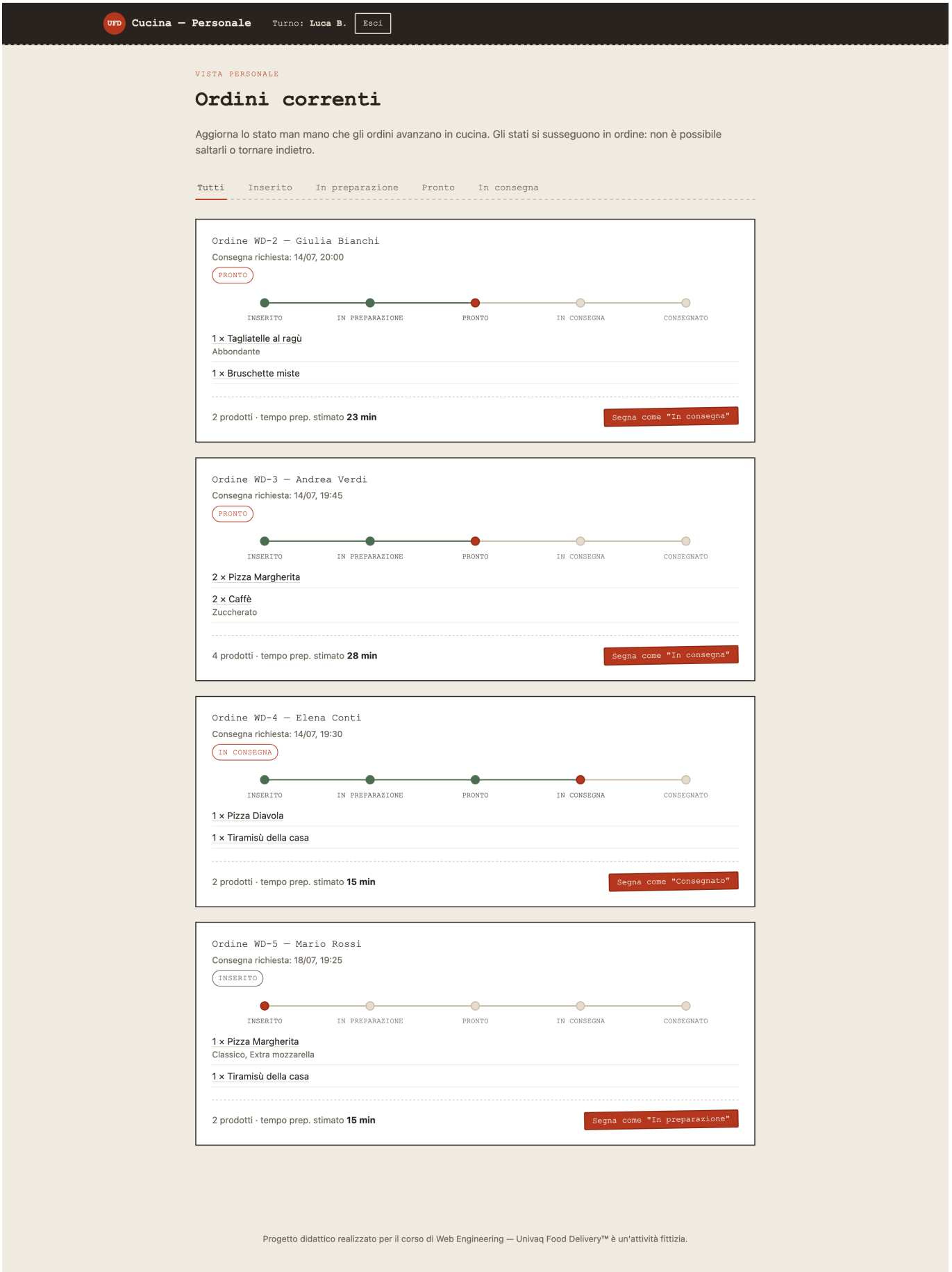


I tuoi ordini



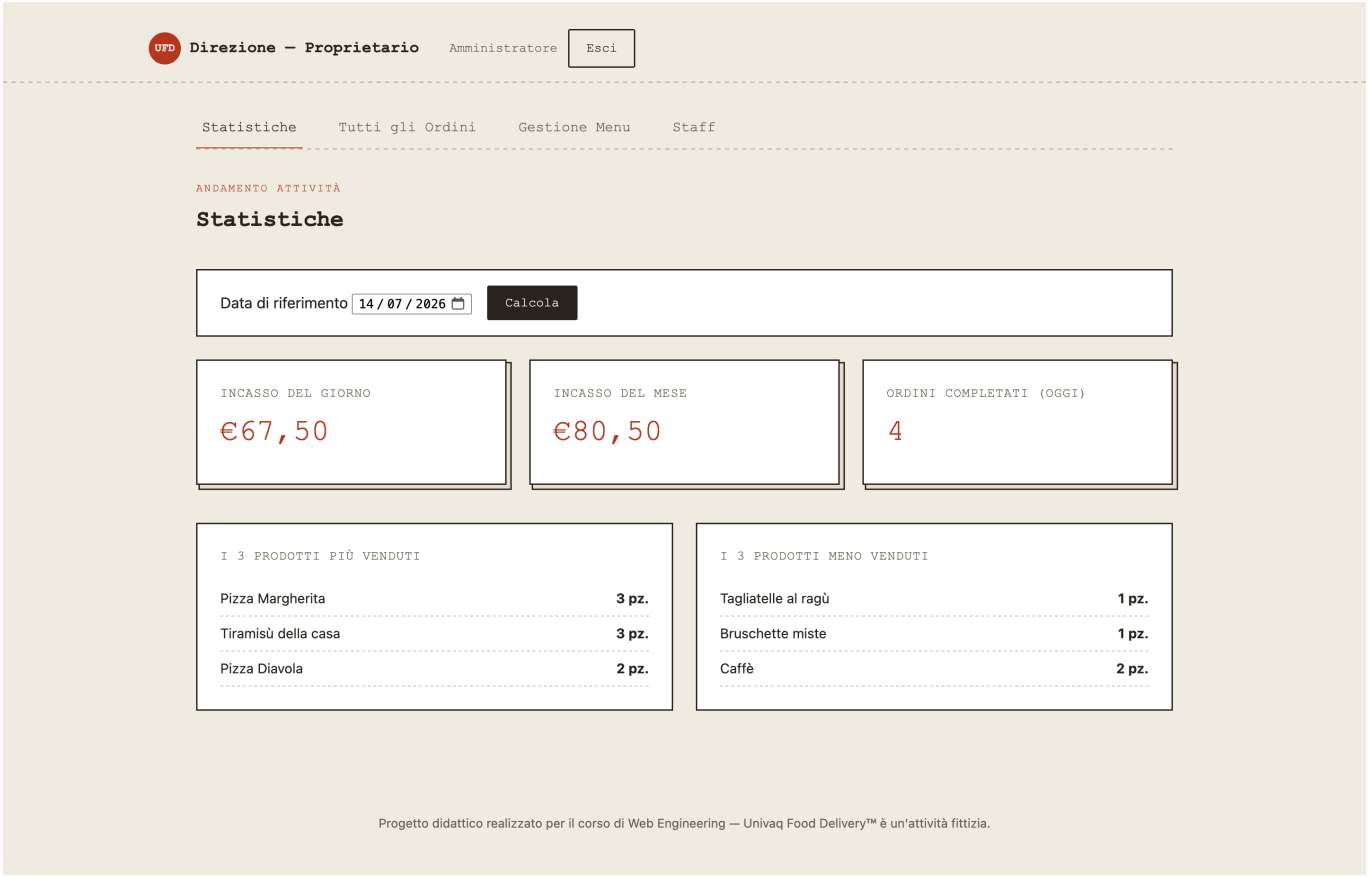
9.3 Schermate personale

Gestione stato ordini

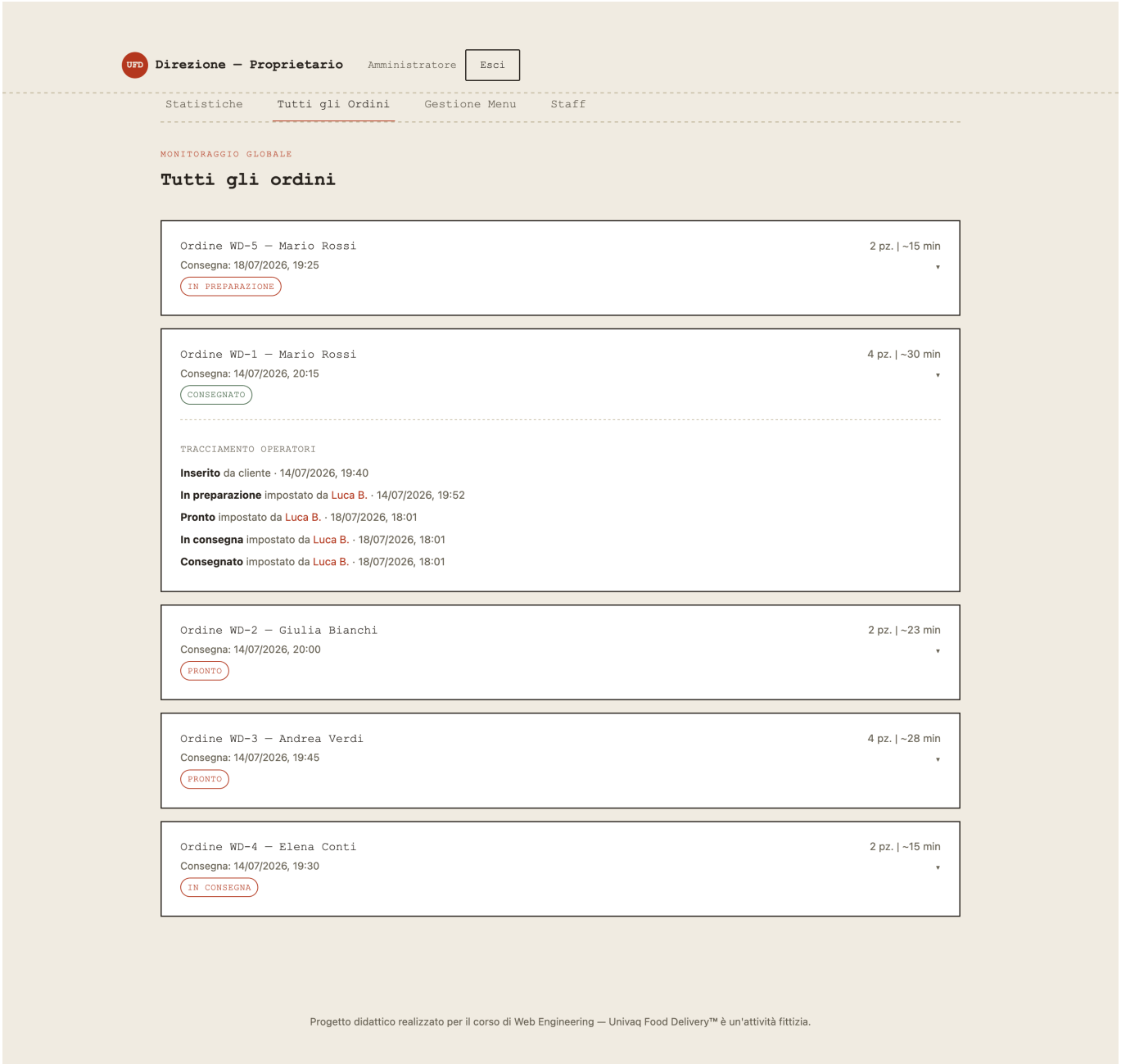


9.4 Schermate proprietario

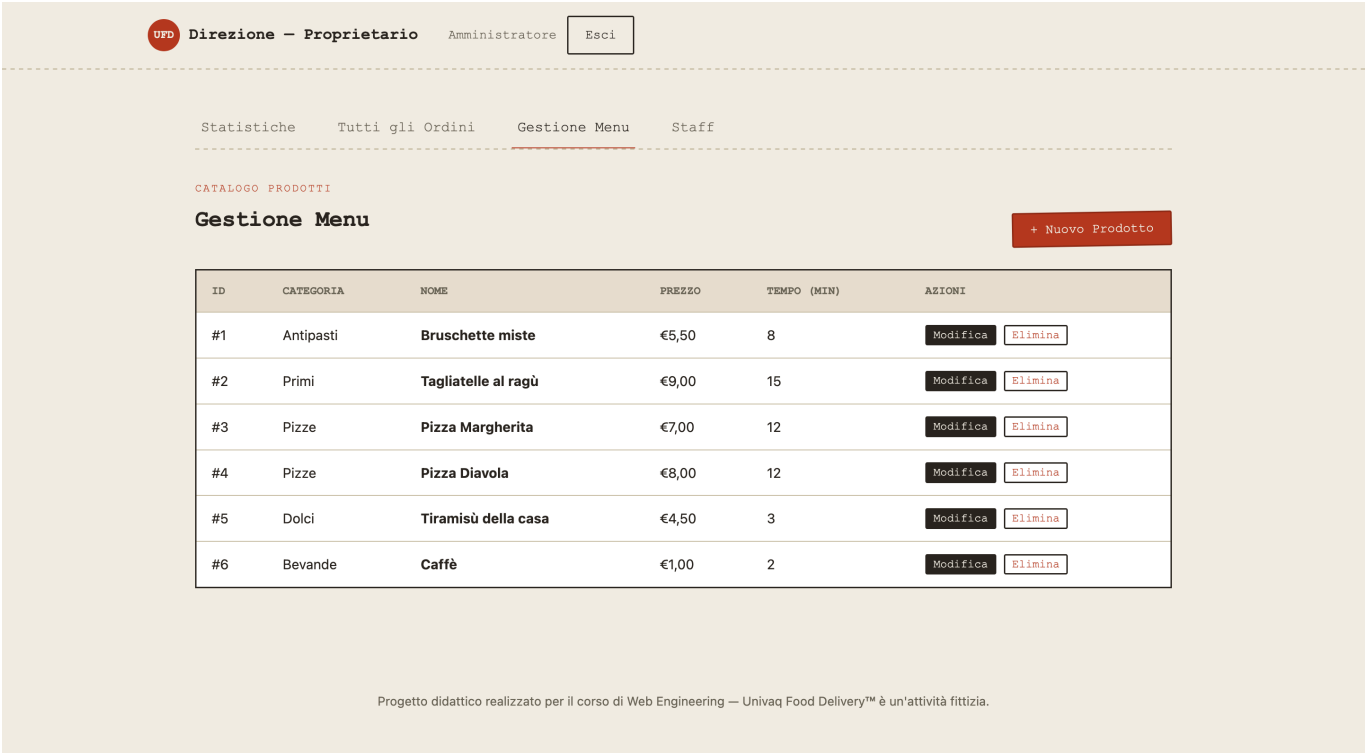
Statistiche



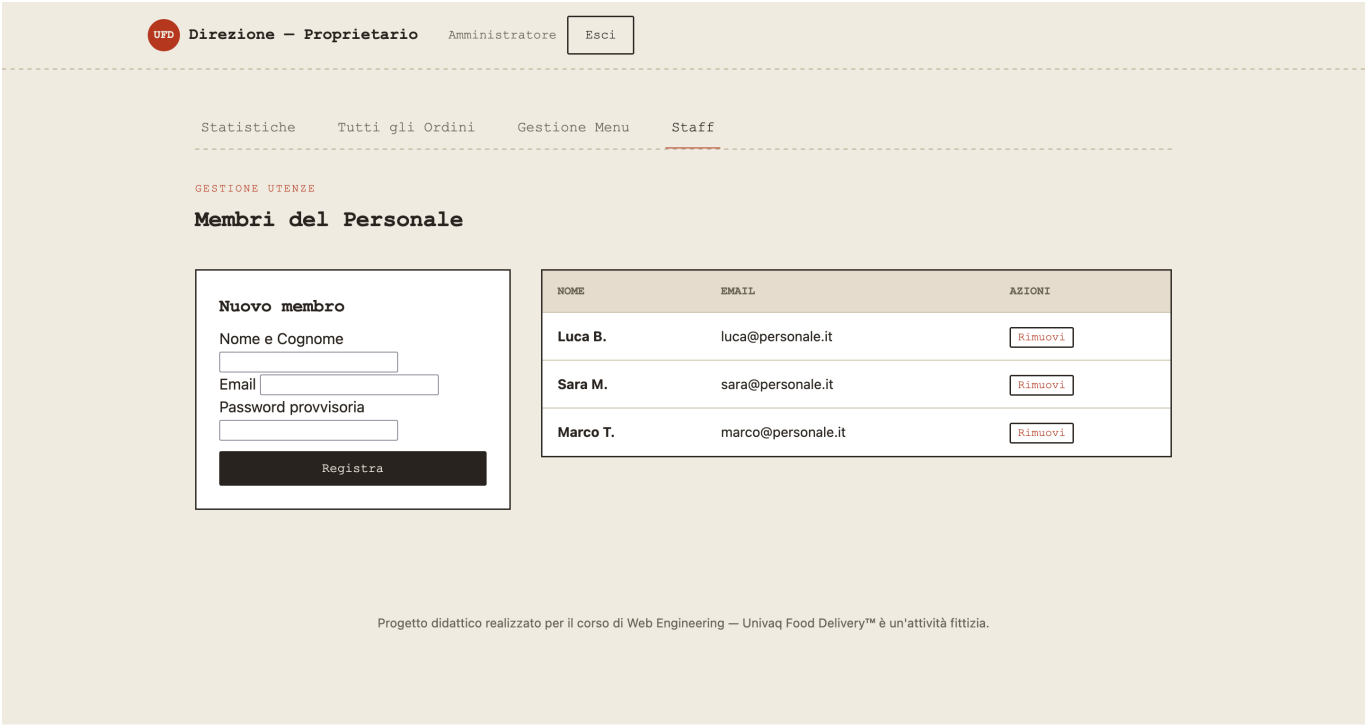
Storico ordini



Gestione menu



Gestione Personale

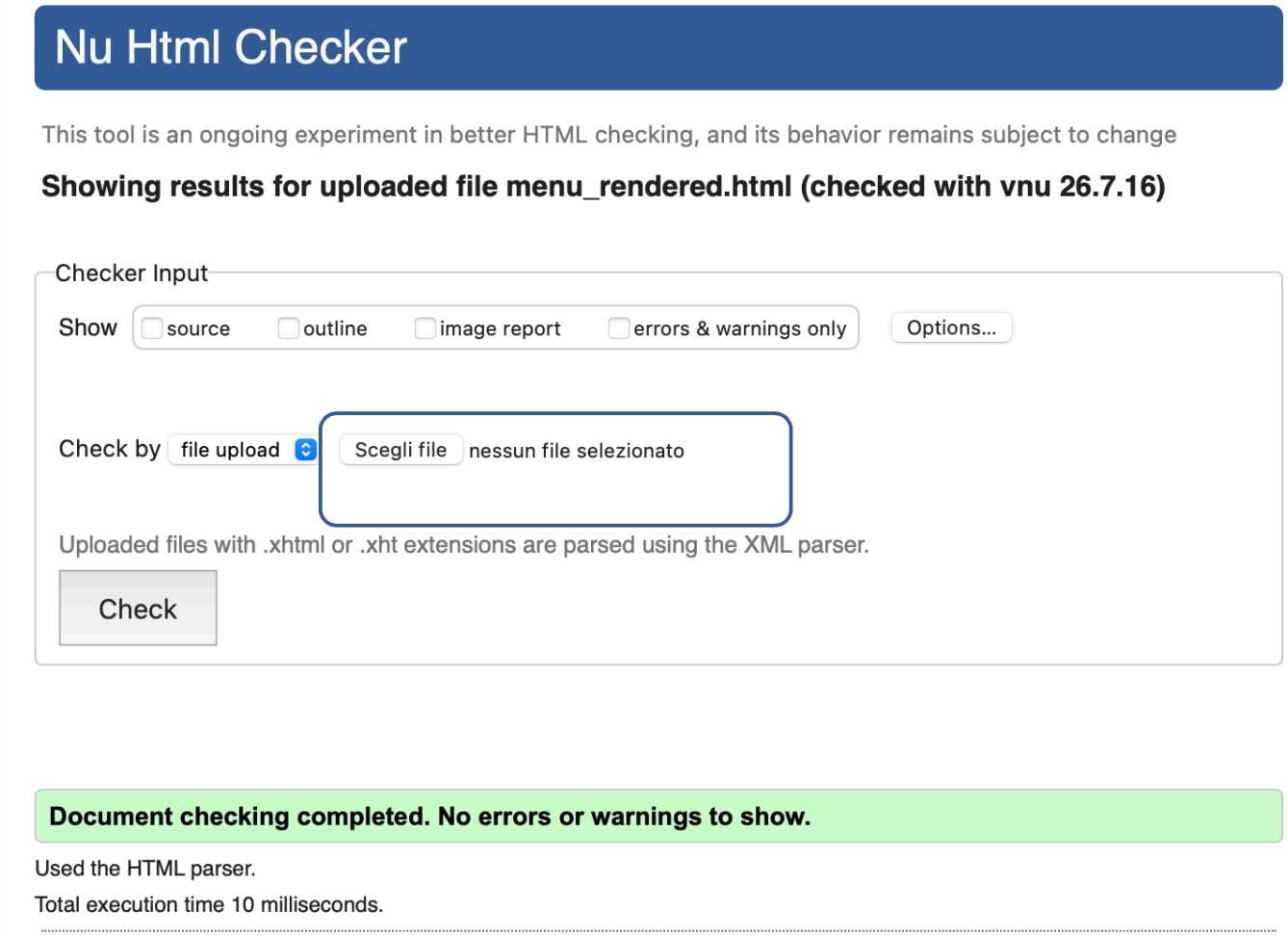


---

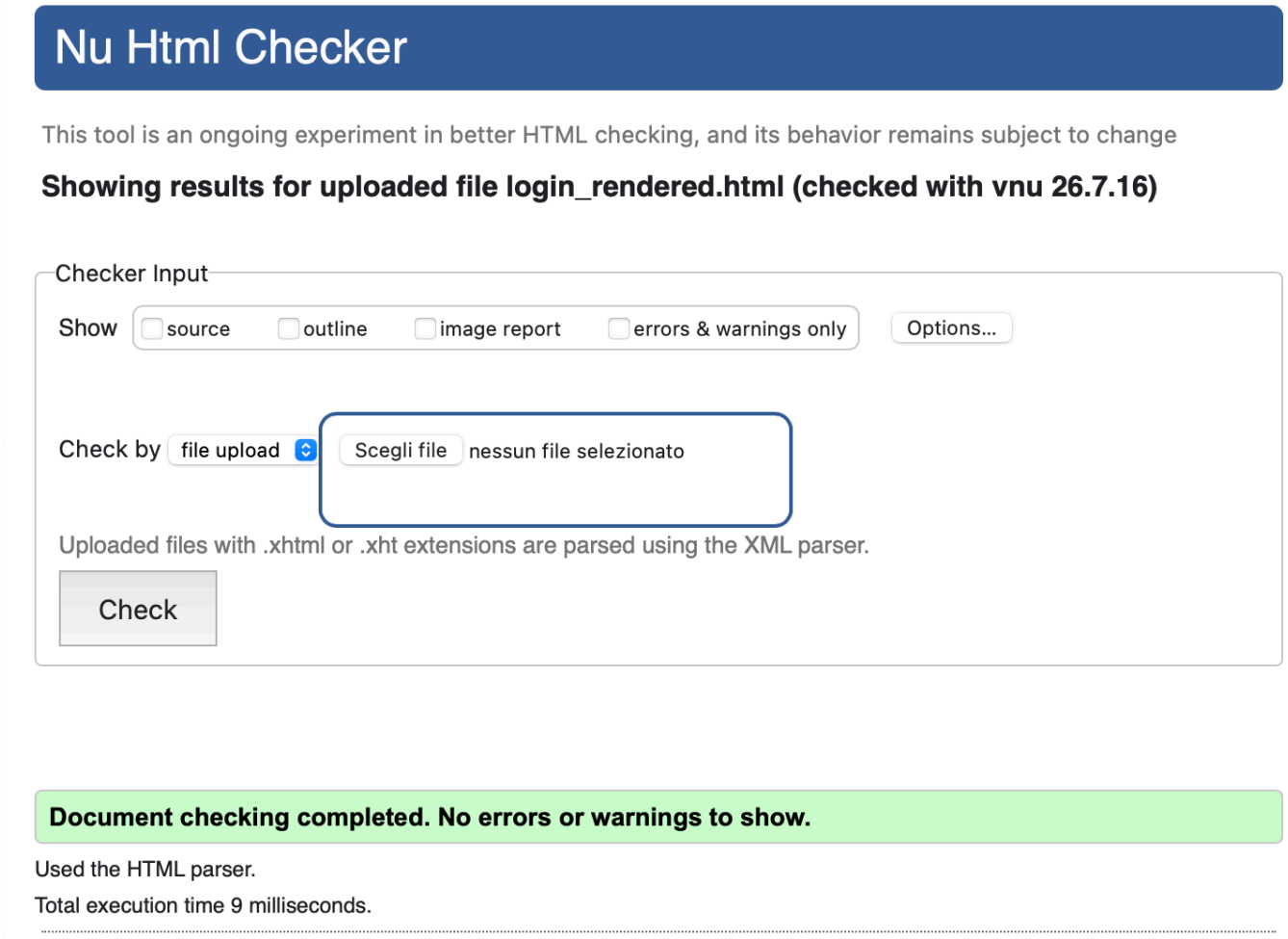
10 Validazione HTML5 (W3C Markup Validation)

Per la validazione **HTML5** abbiamo utilizzato W3C Markup Validator. Abbiamo ricavato i file sorgenti HTML (che abbiamo caricato nella cartella `./docs/HTML5-validator/rendered-html`) che arrivano al browser dalle varie schermate e li abbiamo caricati sul sito. Di seguito i risultati dei vari HTML.

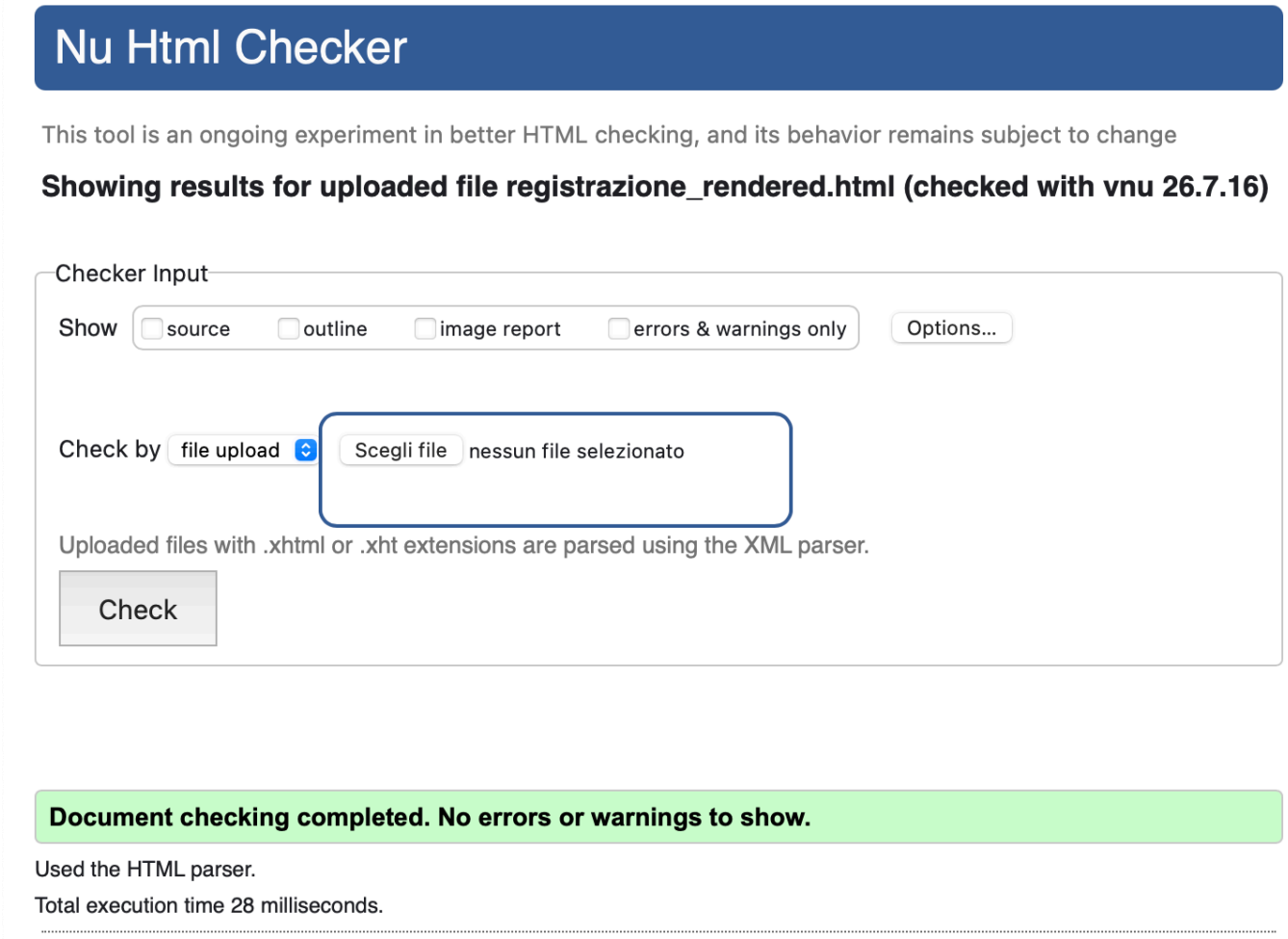
Pagina Menu


Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for uploaded file menu_rendered.html (checked with vnu 26.7.16)
Checker Input
Show
source
outline
image report
errors & warnings only
Options...
Check by
file upload
Scegli file
nessun file selezionato
Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 10 milliseconds.

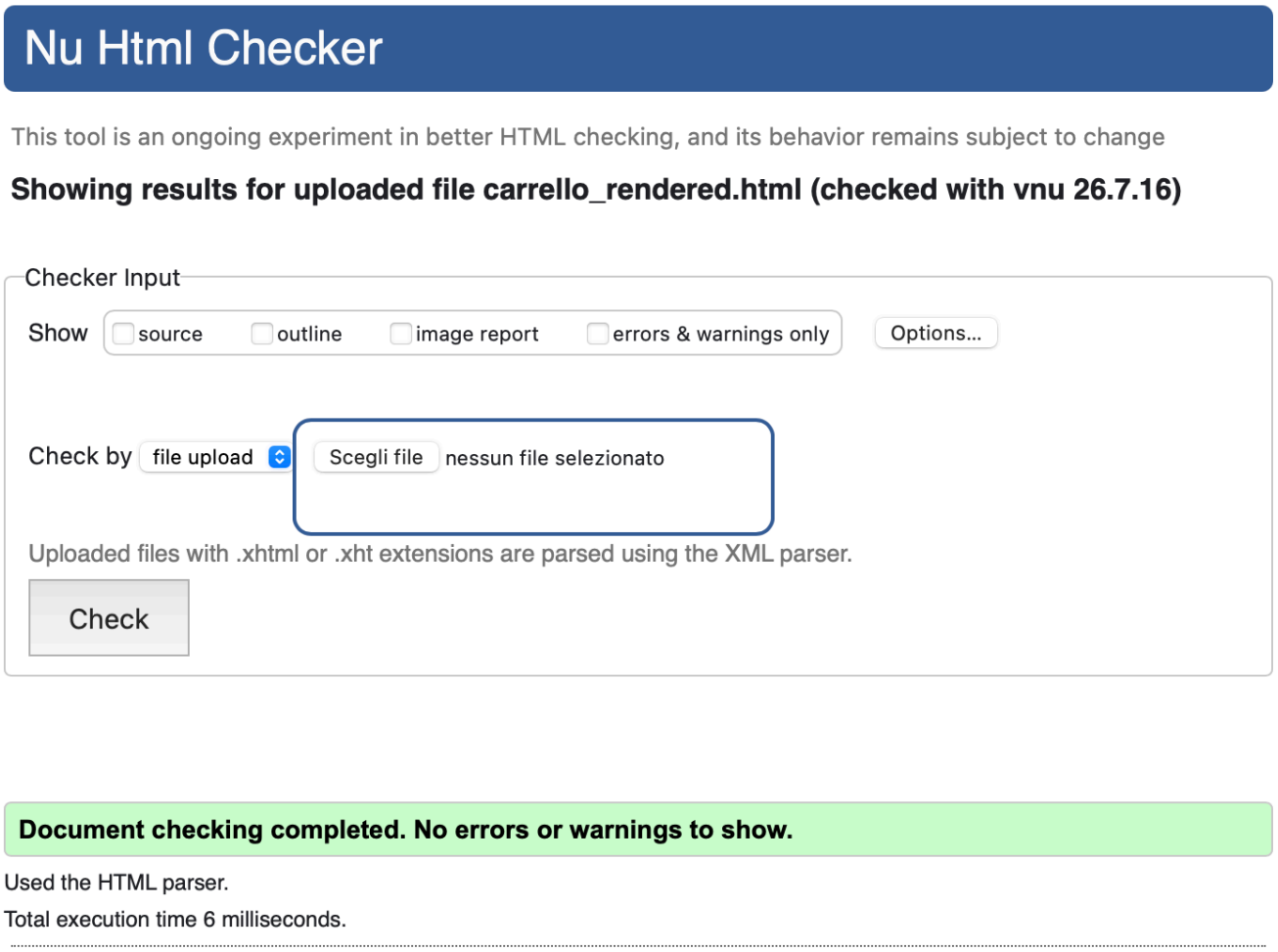
Login


Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for uploaded file login_rendered.html (checked with vnu 26.7.16)
Checker Input
Show
source
outline
image report
errors & warnings only
Options...
Check by
file upload
Scegli file
nessun file selezionato
Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 9 milliseconds.

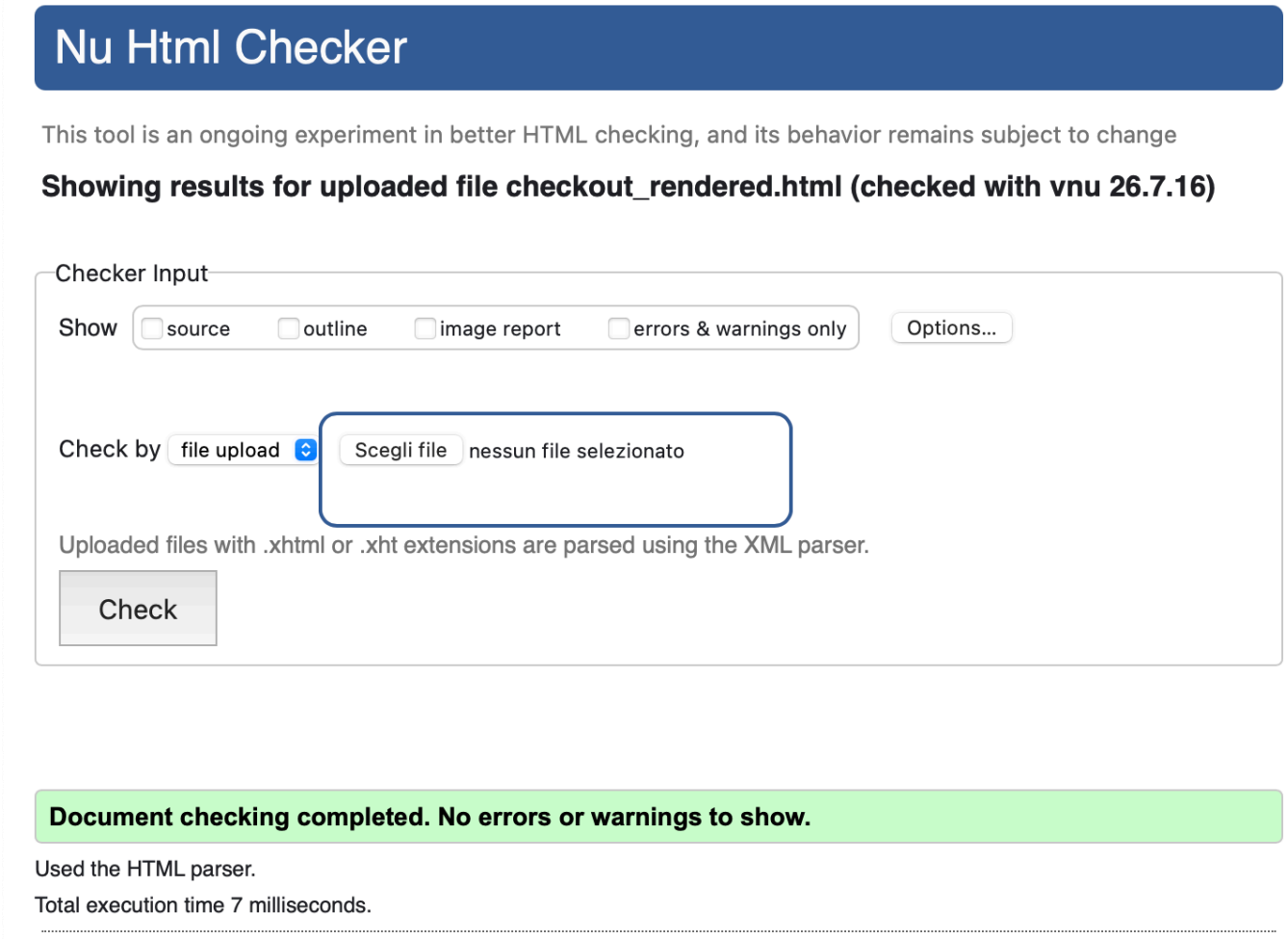
Registrazione



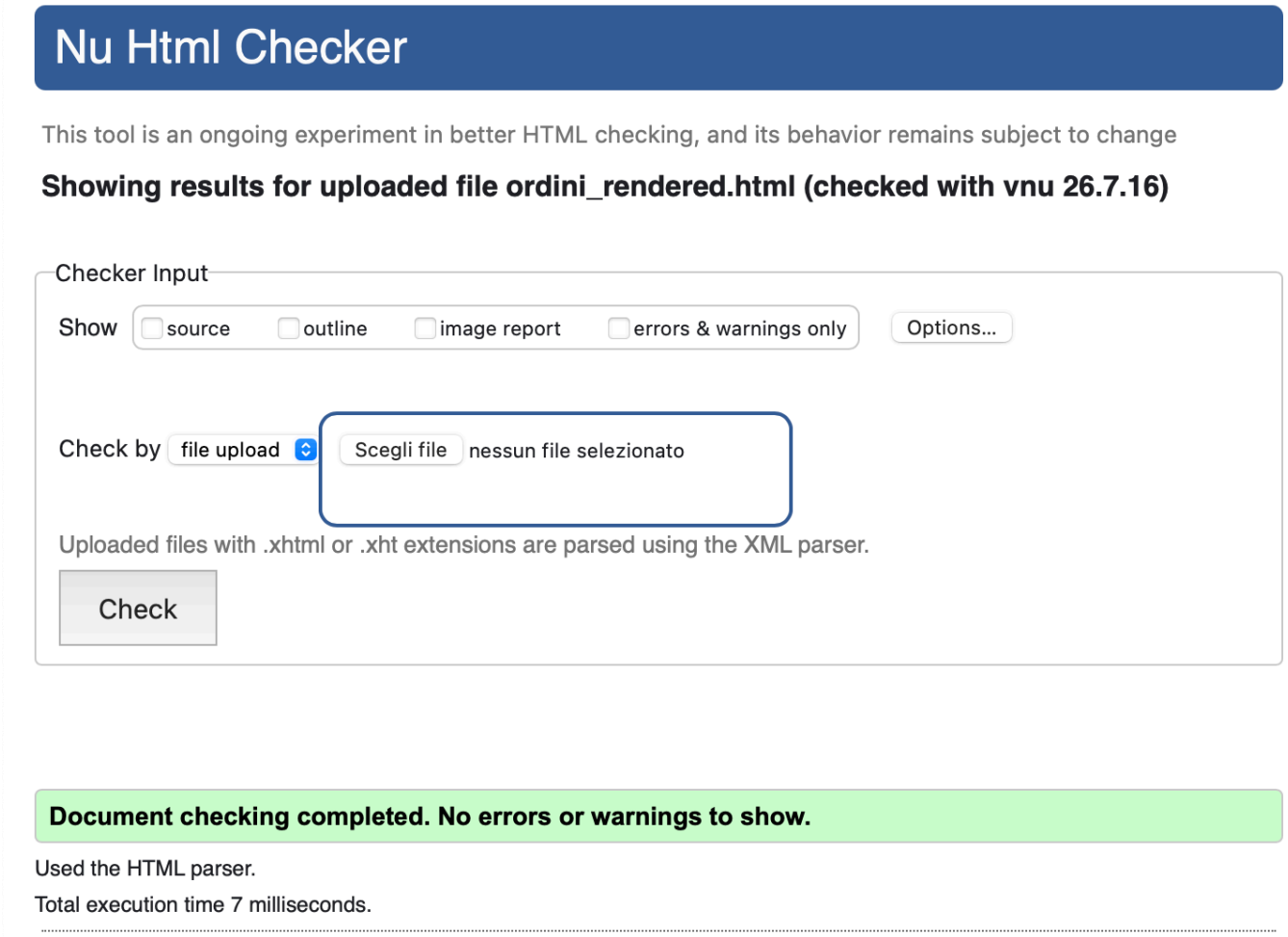
Carrello


Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for uploaded file carrello_rendered.html (checked with vnu 26.7.16)
Checker Input
Show
source
outline
image report
errors & warnings only
Options...
Check by
file upload
Scegli file
nessun file selezionato
Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 6 milliseconds.

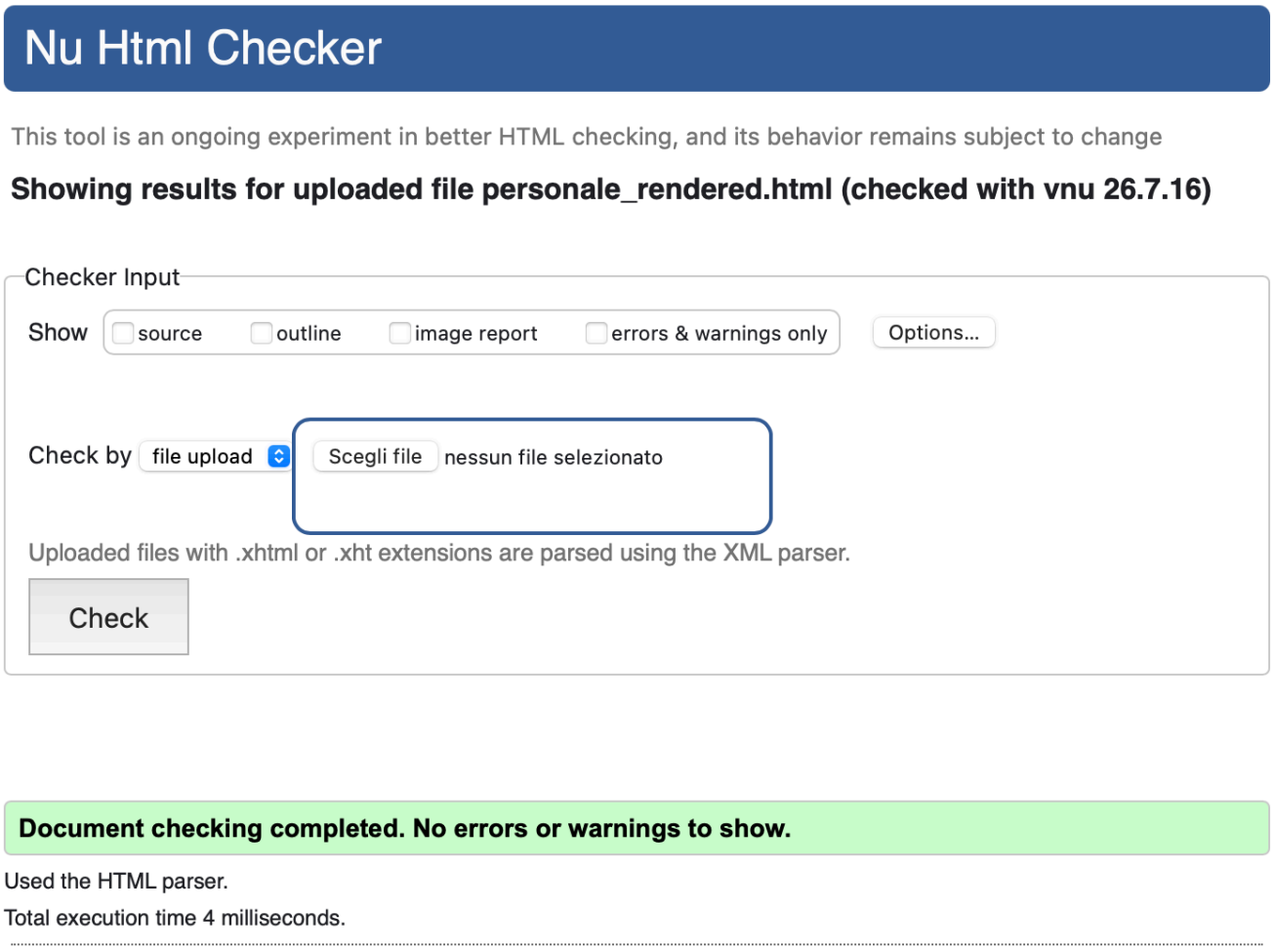
Checkout


Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for uploaded file checkout_rendered.html (checked with vnu 26.7.16)
Checker Input
Show
source
outline
image report
errors & warnings only
Options...
Check by
file upload
Scegli file
nessun file selezionato
Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 7 milliseconds.

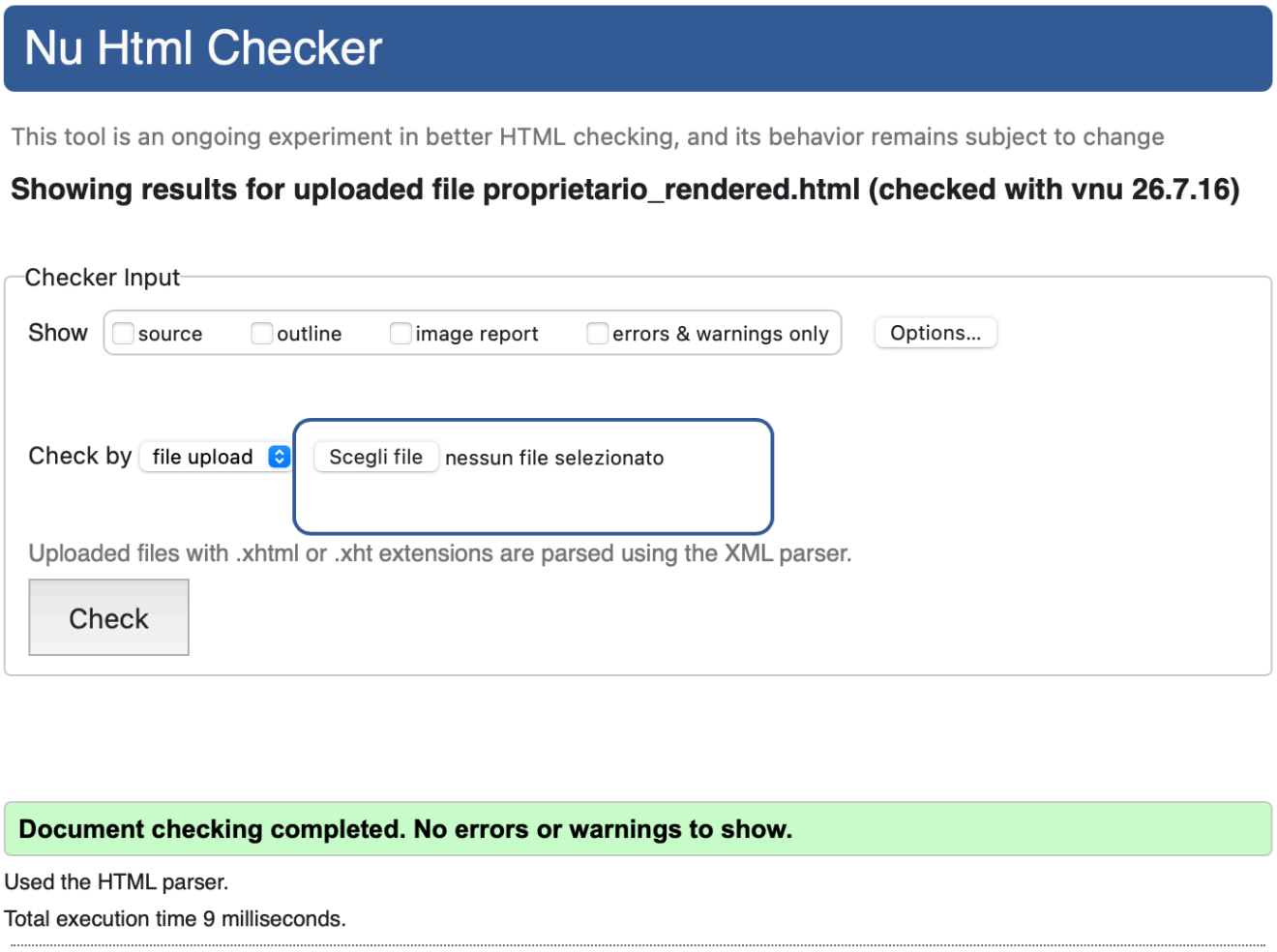
I tuoi ordini



Personale


Nu Html Checker
This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change
Showing results for uploaded file personale_rendered.html (checked with vnu 26.7.16)
Checker Input
Show
source
outline
image report
errors & warnings only
Options...
Check by
file upload
Scegli file
nessun file selezionato
Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.
Check
Document checking completed. No errors or warnings to show.
Used the HTML parser.
Total execution time 4 milliseconds.

Proprietario



---

11. Contributo dei componenti del gruppo

Alessia De Dominicis

- Frontend HTML/CSS/JS
- HTML5 validator
- Hashing password (`PasswordUtil.java`)
- Backend Java Refactor con Gson (API, DTO)
- Controlli/Ricalcoli dati lato server
- Gestione dipendenze
- Possibilità di effettuare un ordine con gli script disattivati (sessione carrello, drawer carrello, ...)
- Fix vari
- Tesing su browser (Safari, Chrome, Opera)
- Documentazione
- Immagini Documentazione

Pier Marco Graziani

- Database
- Backend Java Servlet
- Backend Java Model (Data, DAO)

- Migrazione Frontend da HTML a Freemarker
- `EmailUtil.java`
- Logica per il calcolo delle statistiche del proprietario (`Proprietario.js`)
- Modale per aggiornare/modificare/eliminare prodotti da gestione menu della schermata proprietario
- Fix vari
- Testing su browser (Firefox, Edge)
- Documentazione